IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

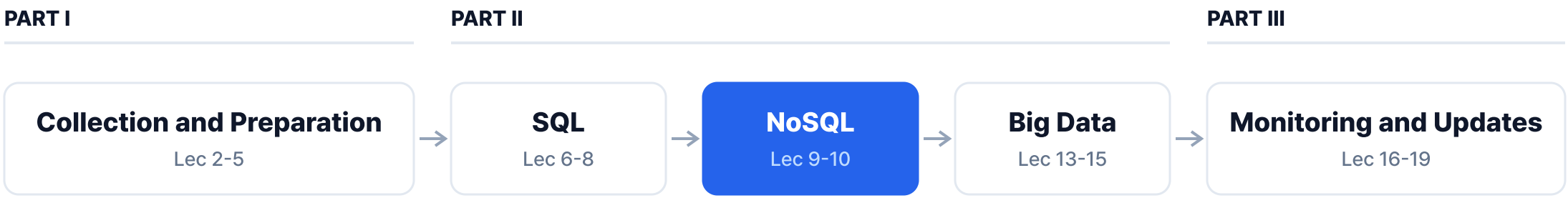
9.

NoSQL I: Landscape

Sangwoo Mo · POSTECH

Industrial & Management Engineering

Where We Are

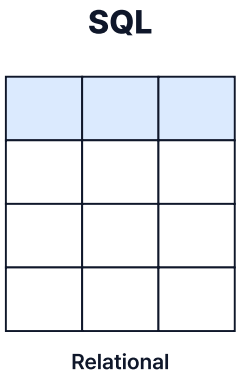

PART I
PART II
PART III
Collection and Preparation
Lec 2-5
SQL
Lec 6-8
NoSQL
Lec 9-10
Big Data
Lec 13-15
Monitoring and Updates
Lec 16-19

Outline: NoSQL I

01	**Four families of NoSQL**	key-value · wide-column · graph · document, with the products you will meet
02	**JSON and semi-structured data**	syntax · types · a tree · self-describing
03	**Between trees and tables**	relations into JSON · JSON into relations · what does not fit · `Demo` pad
04	**Why NoSQL**	data model · schema-less · scale-out and replication · which guarantees to check

Why NoSQL?

Lectures 6 to 8

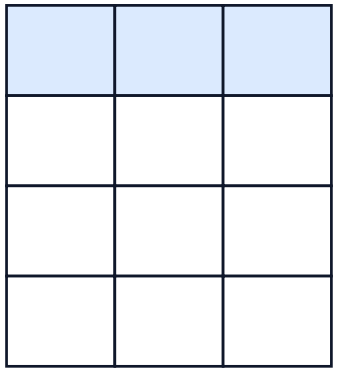


Lectures 6 to 8: one data model, the **relational table**

Why NoSQL?

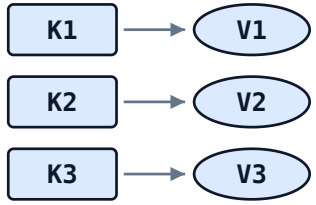
The four NoSQL families

SQL

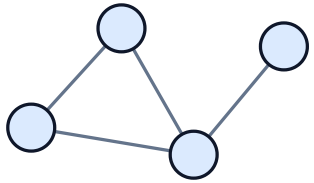


Relational

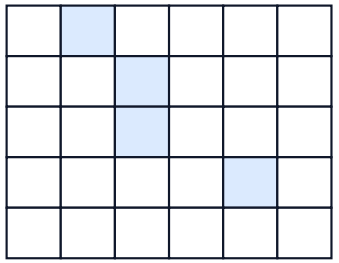
NoSQL



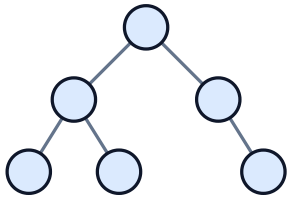
Key-value



Graph



Wide-column



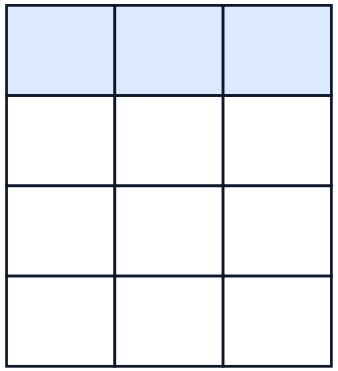
Document

NoSQL is not one thing: **four data models** that are not tables

Key-Value Store

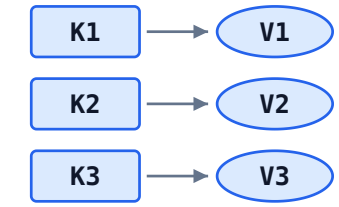
Core idea

SQL

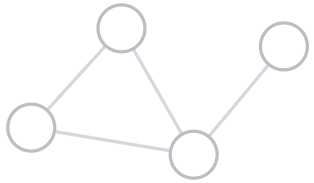


Relational

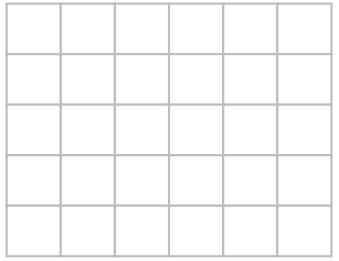
NoSQL



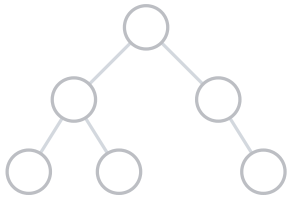
Key-value



Graph



Wide-column



Document

The simplest of the four: a key, and whatever bytes you put under it

Key-Value Store

Example: user records

The key-value model is the simplest NoSQL model: data is stored as **key-value pairs**. It works well for applications that need **fast lookups** by a unique key.

For a **user management system**, user data can be stored as key-value pairs where the key is the `UserID` and the value is the associated user information.

```
Key: 1
Value: { "name": "John", "email": "john@email.com", "age": 28 }

Key: 2
Value: { "name": "Mike", "email": "mike@email.com", "age": 31 }
```

Very efficient for **simple lookups**, but no complex querying or relationships between data.

Key-Value Store

Data model and operations

- **Data model**: (key, value) pairs
  - Key = string/integer, unique for the entire data
  - Value = can be anything (very complex object)
- **Operations**
  - `get(key)`, `put(key, value)`
  - Operations on value not supported
    - user writes their own code for those

Key-Value Store

Exercise: the Product table

Product

PName	Price	Category	Manufacturer
Gizmo	$19.99	Gadgets	GizmoWorks
Powergizmo	$29.99	Gadgets	GizmoWorks
SingleTouch	$149.99	Photography	Canon
MultiTouch	$203.99	Household	Hitachi

How would you represent the **Product** data (Lecture 6) as key-value pairs?

Key-Value Store

Exercise: the Product table

Product

PName	Price	Category	Manufacturer
Gizmo	$19.99	Gadgets	GizmoWorks
Powergizmo	$29.99	Gadgets	GizmoWorks
SingleTouch	$149.99	Photography	Canon
MultiTouch	$203.99	Household	Hitachi

- How would you represent the **Product** data as key-value pairs?
- Option 1: key = `PName`, value = entire product record
- Option 2: key = `Manufacturer`, value = all products of that company
- Option 3: key = `Category`, value = all products in that category

Key-Value Store

Examples: the same lookup pattern serves caches and persistent stores

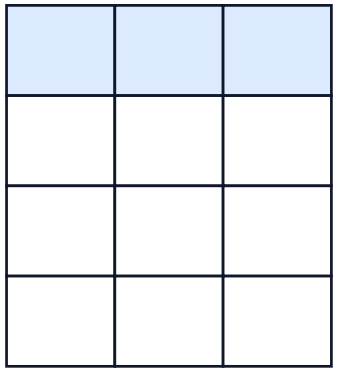
Product / service	Typical role
Redis / Redis Cloud	Sessions, caches, counters, rate limits
Valkey / Amazon ElastiCache	Managed caching and sessions
Memcached	Simple in-memory cache
Amazon DynamoDB	Persistent key-value and document data at scale

Choose by **role and guarantees**: a cache and a system of record differ

Wide-Column Store

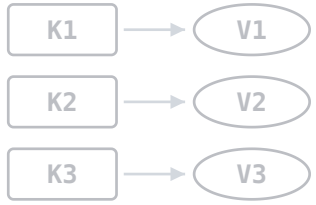
Core idea

SQL

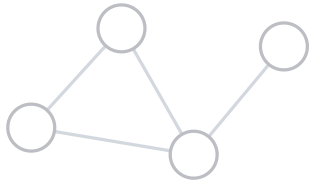


Relational

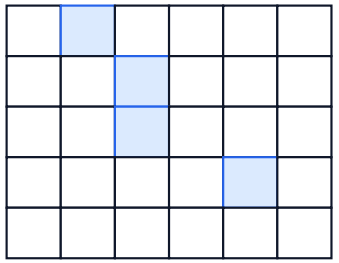
NoSQL



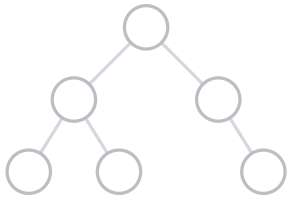
Key-value



Graph



Wide-column



Document

Looks like a table, but every row may have **its own set of columns**

Wide-Column Store

Rows with their own columns

In the column-family model, data is organized into rows and columns, but unlike the relational model, each row can have a **variable number of columns**. It is optimized for **fast querying** and **large-scale distributed storage**.

Example: a **user activity tracking system** that stores the actions users take on a website, such as page views and purchases. Each user has a unique `UserID`, and their activity is stored in a column family.

Column Family: UserActivity

Row Key (UserID)	Page1	Page2	Purchase1	Purchase2
1	Viewed: Home Page	Viewed: Products	Product: Laptop ($1200)	Product: Phone ($800)
2	Viewed: Home Page	Viewed: Contact Us	Product: TV ($1500)	
3	Viewed: Home Page		Product: Camera ($600)	

Wide-Column Store

Example: user activity

Row Key (UserID): the unique identifier for each user.

Page1, Page2, Purchase1, Purchase2: columns representing the user's activity, which can vary from row to row.

- Each user (row) can have a **variable number of columns**, and different users may have different activities stored
- In **Bigtable-style models**, column families are declared; column qualifiers can be added per row
- High write throughput and distributed storage, but it doesn't enforce strict relationships like the relational model

Wide-Column Store

Data model: BigTable and HBase

- Also called wide-column stores
- Based on Google's BigTable
- HBase is an open source implementation of BigTable
- Data model:
  - Variant 1: key = rowID, value = record
  - Variant 2: key = (rowID, columnID), value = field
    - Or multiple columnIDs in the key

Wide-Column Store

Examples: designed for many writes and key-range reads

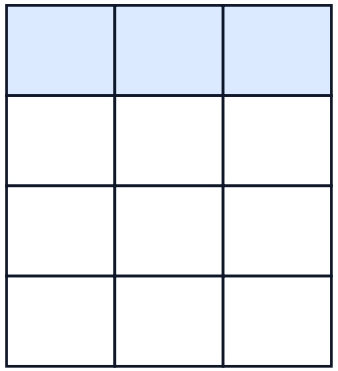
Product / service	What to recognize
Apache Cassandra	Partition key + ordered rows (CQL declares its columns)
Google Cloud Bigtable	Row keys + sparse cells; HBase is its open-source relative
ScyllaDB Cloud	Managed Cassandra-compatible service

Same family, **different concrete models**: check the product's key model

Graph Store

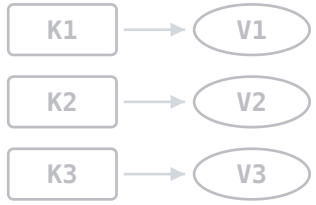
Core idea

SQL

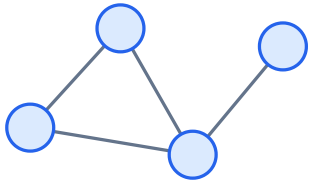


Relational

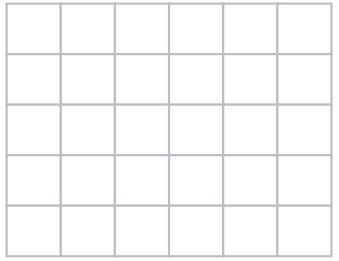
NoSQL



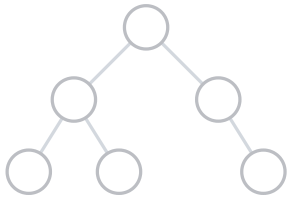
Key-value



Graph



Wide-column

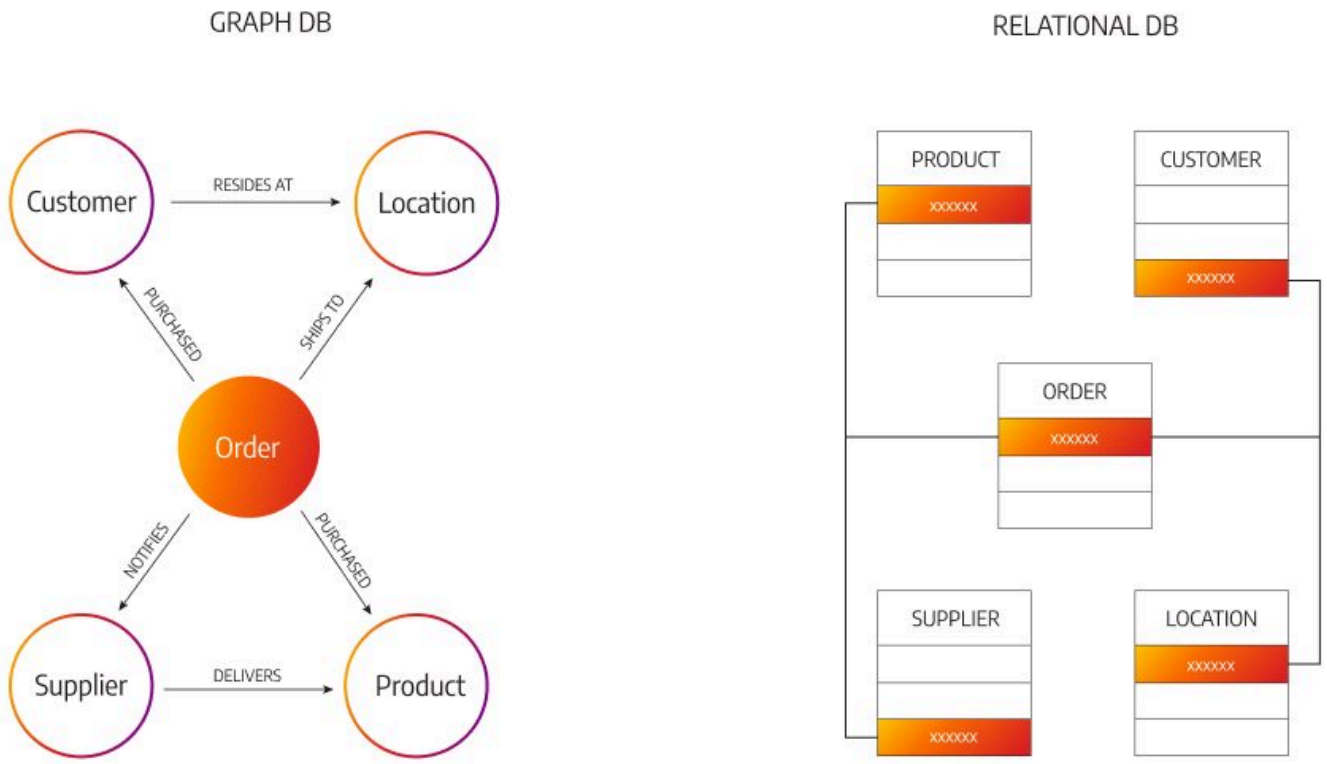


Document

Nodes and edges are the data; **following an edge** replaces a join

Graph Store

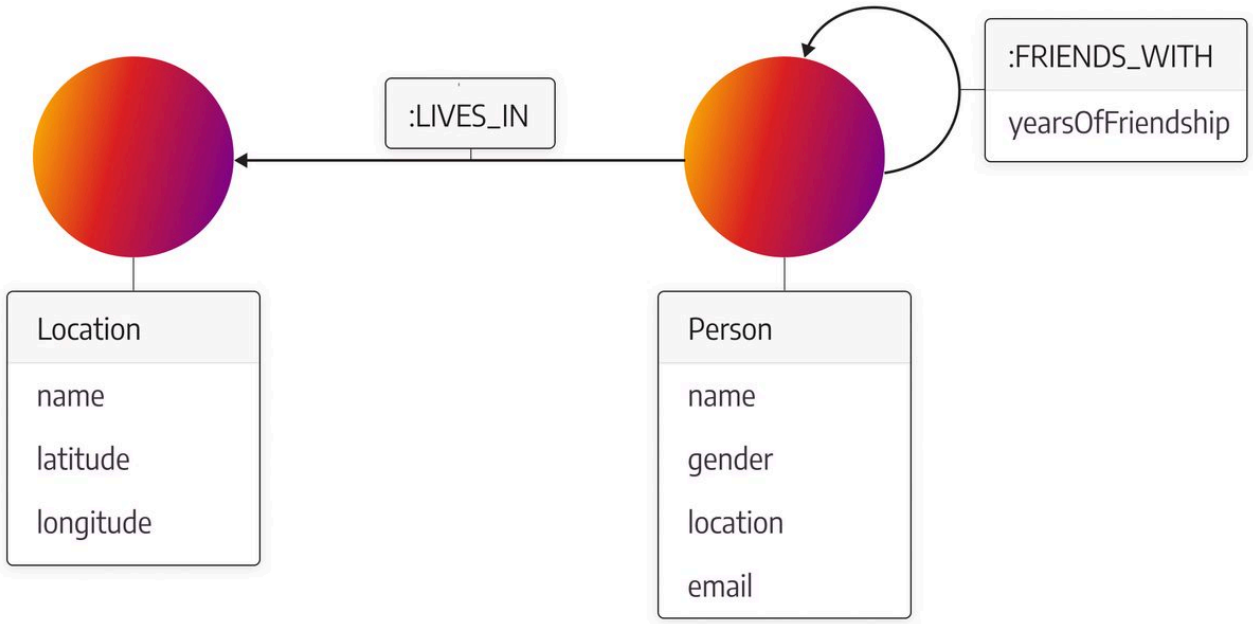
Edges instead of foreign keys



The same five entities: edges on the left, **foreign keys and joins** on the right

Graph Store

Property graphs

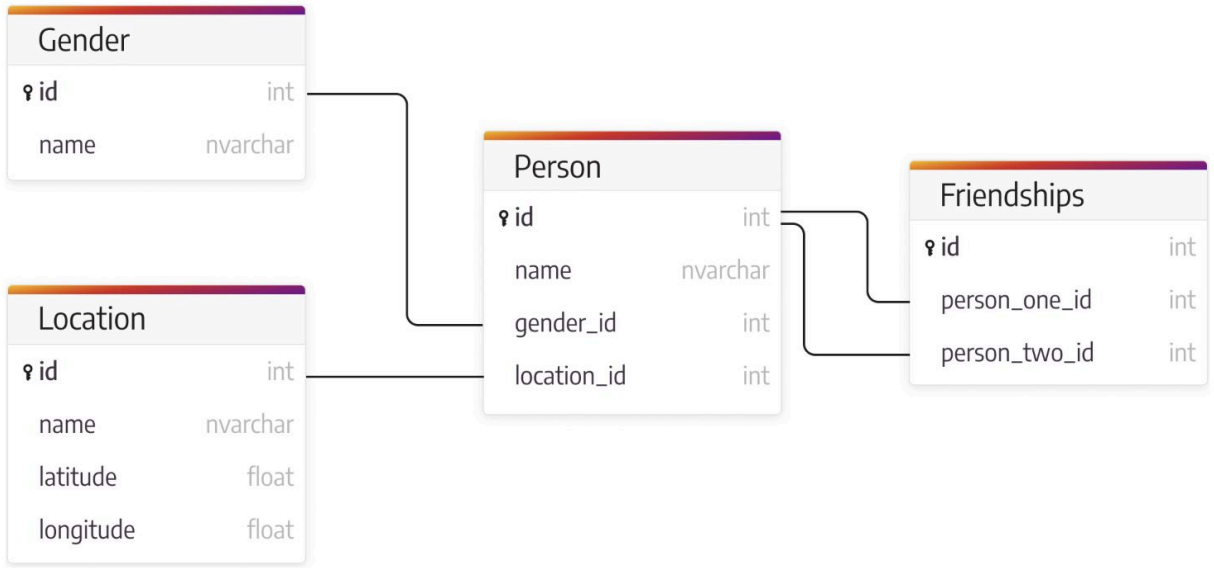


A **property graph**: both nodes and edges carry attributes

Graph Store

When a graph fits

Graph databases represent **relationships as edges** and offer traversal queries. Relational databases can use an edge table with a relationship-type column. The better fit depends on the traversal pattern and workload.



Friendship as a table: two-hop paths use **two copies** of the edge table

Graph Store

Cypher: nodes and edges

```
CREATE (p1:Person {name: 'Harry'}), (p2:Person {name: 'Anna'})
CREATE (p1)-[r:MARRIED_TO]->(p2)
RETURN p1, r, p2;
```



Two nodes and one directed edge, written as a **pattern** rather than as rows

Graph Store

Recursive queries: SQL and Cypher

With breadth-first and depth-first search it is possible to obtain the descendants in a specific order.
Here are the SQL and Cypher queries to **find all descendants** of Thurimbert:

```sql
WITH RECURSIVE descendants AS (
  SELECT person FROM tree
  WHERE  parent = 'Thurimbert'
  UNION ALL
  SELECT t.person
  FROM   descendants d, tree t
  WHERE  t.parent = d.person
)
SELECT * FROM descendants;
```

```cypher
MATCH path = (n:Person {name: 'Thurimbert'})-
[*]->(m)
RETURN m;
```

Any depth: SQL **recurses** one join per level, Cypher writes one path pattern

Graph Store

Examples: chosen when paths of unknown depth are the common question

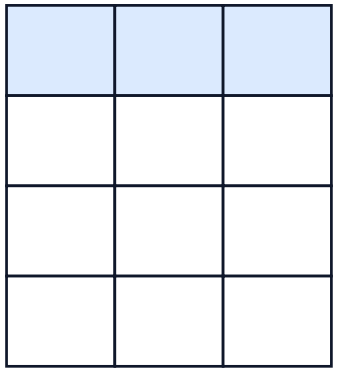
Product / service	Model and query style
Neo4j / AuraDB	Property graph, Cypher (managed: AuraDB)
Amazon Neptune	Property graph and RDF; openCypher, Gremlin, SPARQL
Memgraph	In-memory property graph, Cypher

A graph store fits **multi-hop questions**; a fixed-depth join may not need one

Document Store

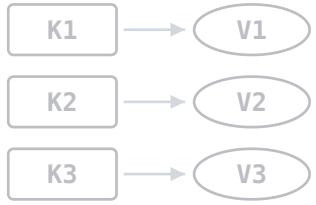
Core idea

SQL

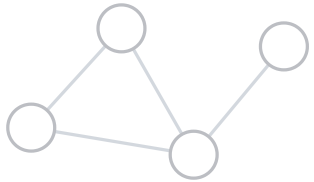


Relational

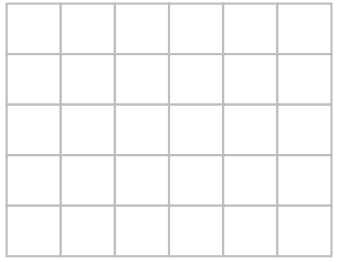
NoSQL



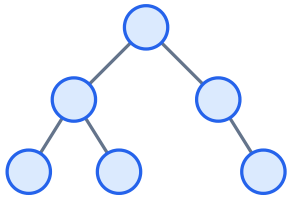
Key-value



Graph



Wide-column



Document

A key-value store whose values have **structure the store can read**

Document Store

Values with structure

- In key-value stores, the value is often a very complex object
  - Key = `'GizmoWorks'`, Value = [all products of that company]
- Better: *value* to be structured data
  - JSON or Protobuf or XML
  - Called a “document” but it’s just data

We will discuss JSON

Document Store

Examples: one document holds what the application reads together

Data	Why a document fits	Example product / service
Product catalog	Fields vary by product	MongoDB / Atlas
User profile	Related fields are read together	MongoDB / Atlas, Couchbase
Web and mobile app records	Objects in the shape the app reads	Cloud Firestore
JSON inside a relational DB	Mixed with tables and SQL	PostgreSQL jsonb, SQLite JSON

JSON data can also live **inside a SQL database**

JSON Overview

- JavaScript Object Notation = lightweight text-based open standard designed for human-readable data interchange. Interfaces in C, C++, Java, Python, Perl, etc.
- The filename extension is `.json`.

We will emphasize JSON as **semi-structured** data

JSON Syntax

```
{ "book": [
    { "id": "01",
      "language": "Java",
      "author": "H. Javeson",
      "year": 2015
    },
    { "id": "07",
      "language": "C++",
      "edition": "second",
      "author": "E. Sepp",
      "price": 22.25
    }
  ]
}
```

JSON vs Relational

- Relational data model
  - Rigid flat structure (tables)
  - Schema must be fixed in advance
  - Storage encoding is a separate design choice
  - Query language based on Relational Algebra
- Semistructured data model / JSON
  - Flexible, nested structure (trees)
  - Does not require predefined schema (“self-describing”)
  - JSON text is an interchange format; stores may use binary encodings
  - Queries via MQL, SQL/JSON, and other languages

JSON Types

- Primitive: number, string, Boolean, null
- Object: collection of name-value pairs:
  - `{"name1": value1, "name2": value2, …}`
  - “name” is also called a “key”
- Array: *ordered* list of values:
  - `[obj1, obj2, obj3, ...]`

Avoid Using Duplicate Keys

The standard allows them, but many implementations don’t

```
{ "id": "07",
  "title": "Databases",
  "author": "Garcia-Molina",
  "author": "Ullman",
  "author": "Widom"
}
```

same key again



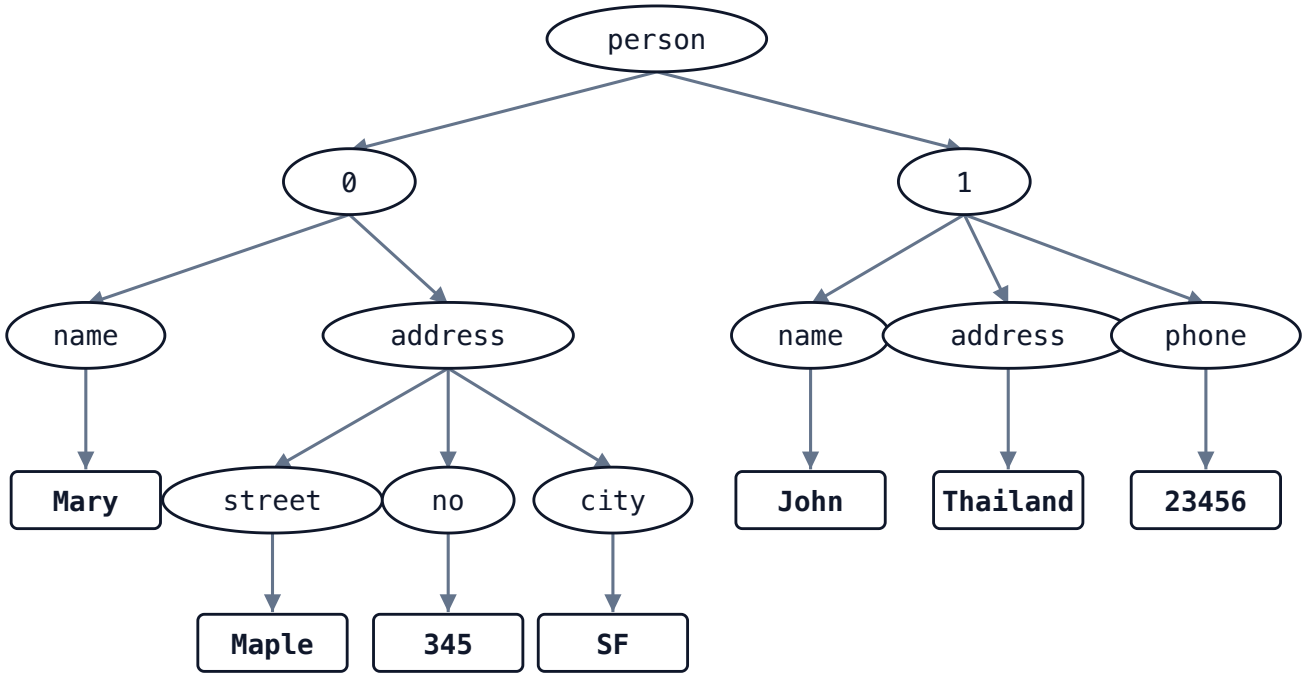
Use an ordered list instead

```
{ "id": "07",
  "title": "Databases",
  "author": ["Garcia-
Molina",
             "Ullman",
             "Widom"]
}
```

one key, an array

JSON Semantics: A Tree!

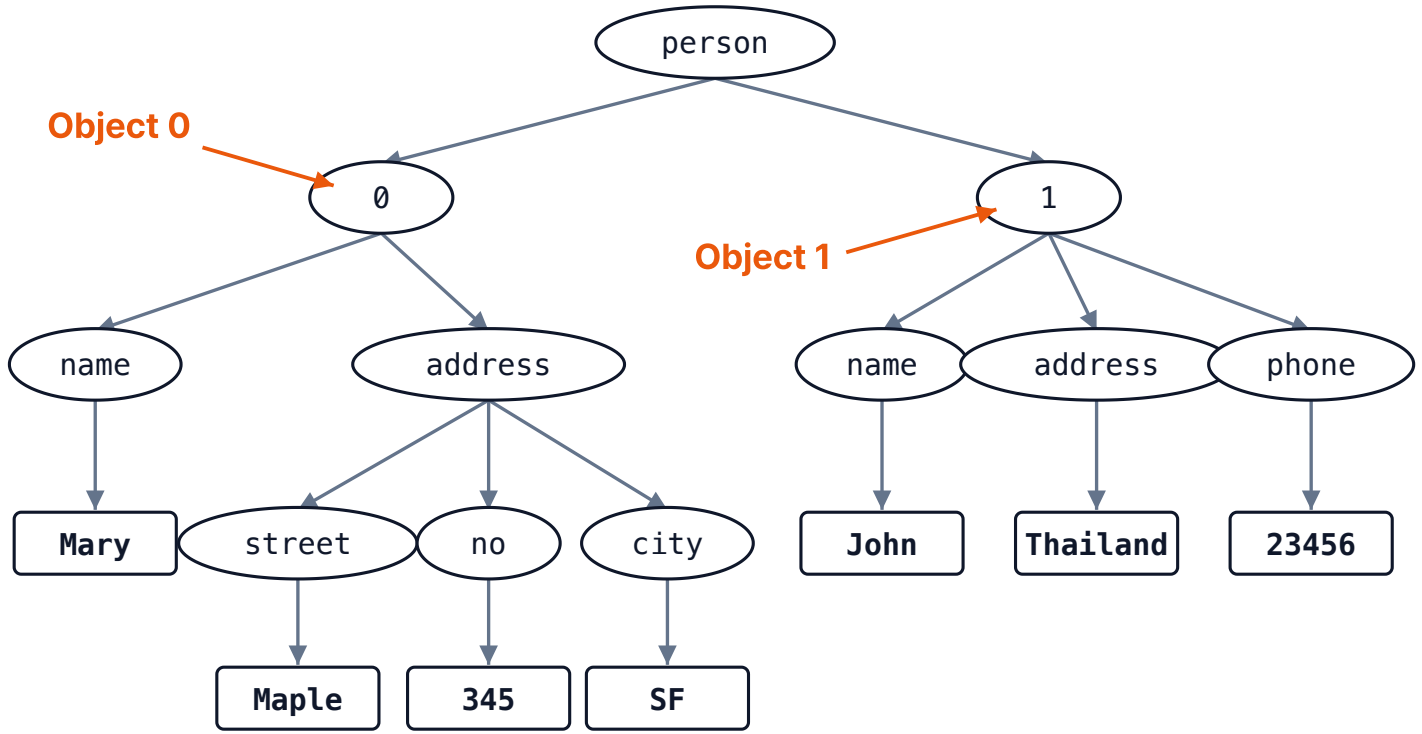
```
{"person":
  [ {"name": "Mary",
     "address":
       {"street": "Maple",
        "no": 345,
        "city": "SF"}},
    {"name": "John",
     "address": "Thailand",
     "phone": 23456}
  ]
}
```



Every value sits at the end of **one path** from the root

JSON Semantics: A Tree!

```
{"person":
  [ {"name": "Mary",
     "address":
       {"street": "Maple",
        "no": 345,
        "city": "SF"}},
    {"name": "John",
     "address": "Thailand",
     "phone": 23456}
  ]
}
```



Recall: arrays are *ordered* in JSON!

Array positions 0 and 1 label edges too, so **arrays keep their order**; relations do not

Semi-Structured Data

- JSON is **self-describing**
- Schema elements become part of the data
  - Relational schema: `person(name, phone)`
  - In JSON `"person"`, `"name"`, `"phone"` are part of the data, and are repeated many times
- ⇒ JSON is more flexible
  - Schema can change per tuple

Storing JSON in RDBMS

- Using JSON as a data type provided by RDBMSs
  - Declare a column that contains either `json` or `jsonb` (binary)
    - `CREATE TABLE people (person json)` [or `jsonb` for binary]
  - In our previous example, we will have one row per person
    - i.e., a row corresponding to that person's attributes
  - Queries now mix relational and semi-structured syntax
    - `SELECT * FROM people WHERE person @> '{"name": "Mary"}';`
- Translate JSON documents into relations

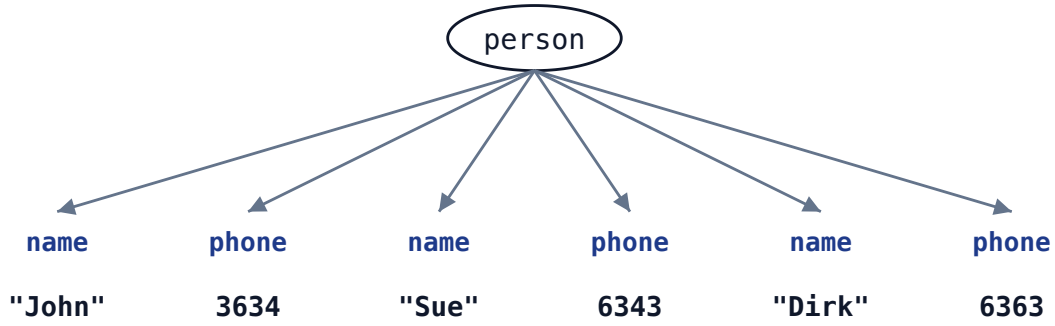
Mapping Relational Data to JSON

One table

Person

name	phone
John	3634
Sue	6343
Dirk	6363

```
{"person": [
  {"name": "John", "phone": 3634},
  {"name": "Sue",  "phone": 6343},
  {"name": "Dirk", "phone": 6363}
]}
```



A table is an array of objects: **one object per row**, one key per column

Mapping Relational Data to JSON

Inlining a foreign key

May inline multiple relations based on foreign keys

Person

name	phone
John	3646
Sue	6343

Orders

personName	date	product
John	2002	Gizmo
John	2004	Gadget
Sue	2002	Gadget

```
{"Person": [
  {"name": "John", "phone": 3646,
   "Orders": [
     {"date": 2002, "product": "Gizmo"},
     {"date": 2004, "product": "Gadget"}
   ]},
  {"name": "Sue", "phone": 6343,
   "Orders": [
     {"date": 2002, "product": "Gadget"}
   ]}
]}
```

orders nested inside the person

One-to-many nests cleanly: the join is **done once, at write time**

Mapping Relational Data to JSON

Many-to-many relationships

Many-many relationships are more difficult to represent

Person

name	phone
John	3634
Sue	6343

Orders

personName	date	product
John	2002	Gizmo
John	2004	Gadget
Sue	2002	Gadget

Product

prodName	price
Gizmo	19.99
Phone	29.99
Gadget	9.99

Options for the JSON file:

- 3 flat relations: Person, Orders, Product
- Person → Orders → Products: products are duplicated
- Product → Orders → Person: persons are duplicated

With many-to-many, **every nesting direction copies something**

Caveat: Stale Copies

Two documents cache the current catalog price, but only one copy receives the update

John's Document

```
{"name": "John",
 "Orders": [
   {"date": 2004,
    "product": {"prodName": "Gadget",
                "currentCatalogPrice": 12.99}}
 ]}
```

Sue's Document

```
{"name": "Sue",
 "Orders": [
   {"date": 2002,
    "product": {"prodName": "Gadget",
                "currentCatalogPrice": 9.99}}
 ]}
```

Remedy: reference the current price or maintain every cached copy

One product, **two prices**: copies need an owner who updates them all

Mapping Semi-Structured Data to Relations

Missing attributes

- Missing attributes:

```
{"person":
  [{"name": "John", "phone": 1234},
   {"name": "Joe"}]          // no phone!
}
```

- Could represent in a table with nulls

name	phone
John	1234
Joe	NULL

Mapping Semi-Structured Data to Relations

Repeated attributes

- Repeated attributes

```
{"person":
  [{"name": "John", "phone": 1234},
   {"name": "Mary", "phone": [1234, 5678]}]   // two phones!
}
```

- Not in one scalar phone column:

name	phone		
Mary	2345	3456	???

Mapping Semi-Structured Data to Relations

Attributes with mixed types

- Attributes with different types in different objects

```
{"person":
  [{"name": "Sue", "phone": 3456},
   {"name": {"first": "John", "last": "Smith"},   // structured name!
    "phone": 2345}]
}
```

- Nested collections
- Heterogeneous collections
- **These are difficult to represent in the relational model**

Recap: The People Table

The four people of the last three slides, as rows of one SQLite table. The whole document sits in a text column; SQLite reads it with its JSON functions.

id	person
1	{"name": "John", "phone": 1234}
2	{"name": "Mary", "phone": [1234, 5678]}
3	{"name": "Joe"}
4	{"name": {"first": "John", "last": "Smith"}, "phone": 2345}

One row per person; the **document is a column value**

Demo: JSON Inside a SQL Table

Storing JSON in RDBMS, made runnable (SQLite JSON functions)

1. Each person's name: `person ->> '$.name'`. What comes back for row 4?
2. One row per phone number: `FROM people p, json_each(p.person, '$.phone') j`. Who disappears, who doubles?
3. People with no phone at all: `json_type(person, '$.phone') IS NULL`.

```
-- write your query here, then Run
```

QUERY RESULT

Write a query on the left, then Run.

Answer: JSON Inside a SQL Table

```
SELECT p.id, p.person ->> '$.name' AS name,
j.value AS phone
FROM   people p, json_each(p.person, '$.phone') j;
```

id	name	phone
1	John	1234
2	Mary	1234
2	Mary	5678
4	{"first":"John","last":"Smith"}	2345

`json_each` turns the phone array into rows: Mary twice, Joe not at all. The relational answer to a repeated attribute is **a second table**, built here on the fly.

A repeated attribute becomes **one row per value**: the join is back

Discussion: Why Semi-Structured Data?

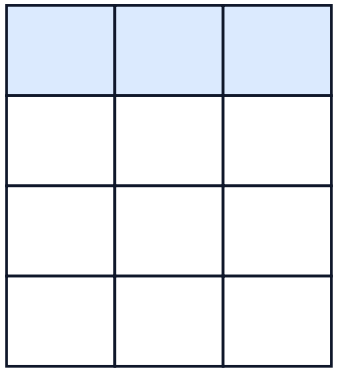
- Semi-structured data works well as *data exchange formats*
  - i.e., exchanging data between different apps
  - Examples: XML, JSON, Protobuf (protocol buffers)
- Increasingly, systems use them as a data model for DBs:
  - SQL Server supports XML-valued relations
  - CouchBase, MongoDB, Snowflake: JSON
  - Dremel (BigQuery): Protobuf

Semi-Structured Data Model

- Nested formats: JSON, XML; Protobuf requires a message schema
- Data model = Tree
- Query language take non first normal form into account as we will see
    - Various “extra” constructs introduced as a result
        - Nesting & Unnesting, strict aggregates, splitting

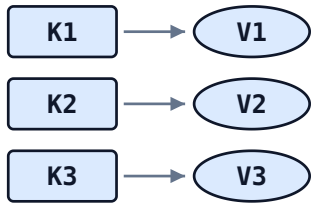
Why NoSQL? – Data Model

SQL

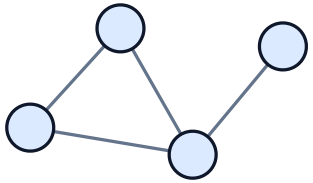


Relational

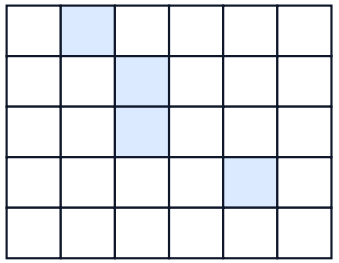
NoSQL



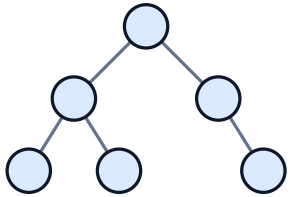
Key-value



Graph



Wide-column



Document

First answer: the data is **not shaped like a table**

Why NoSQL? – Schema-less

SQL: schema first

Let’s take the example of a **user management system**. In SQL, we first define the structure (schema) of the `Users` and `Orders` tables before adding data.

```
CREATE TABLE Users (
  UserID INT PRIMARY KEY,
  Name   VARCHAR(100),
  Email  VARCHAR(100),
  Age    INT
);
```

```
CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  UserID  INT,
  Product VARCHAR(100),
  Price   DECIMAL(10, 2),
  FOREIGN KEY (UserID) REFERENCES
Users(UserID)
);
```

Why NoSQL? – Schema-less

Documents: mixed shapes

Document databases can allow **mixed record shapes**. Adding a field need not rewrite every document, but readers, validation rules, and old records may still need migration.

Example: User and Orders in a Document Database (e.g., MongoDB)

In a document-based NoSQL database, you store user and order data in a single document.

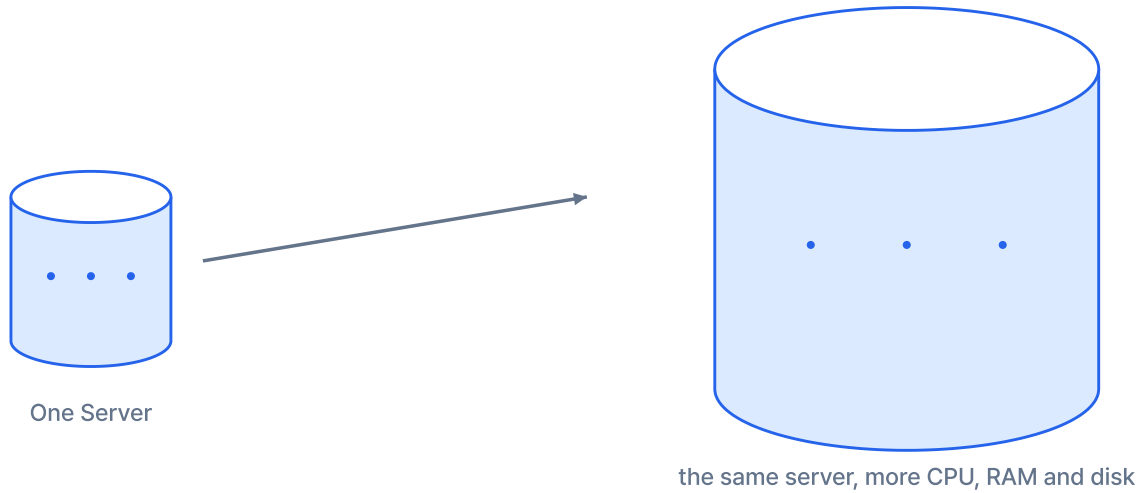
```
{
  "_id": 1,
  "name": "John",
  "email": "john@email.com",
  "age": 28,
  "orders": [
    { "orderId": 101,
      "product": "Laptop",
      "price": 1200 },
    { "orderId": 104,
      "product": "Smartphone",
      "price": 800 }
  ]
}
```

orders live inside the user

Why NoSQL? – Scalability

Scale up

A single-server system can scale **vertically** (also known as **scale-up**).



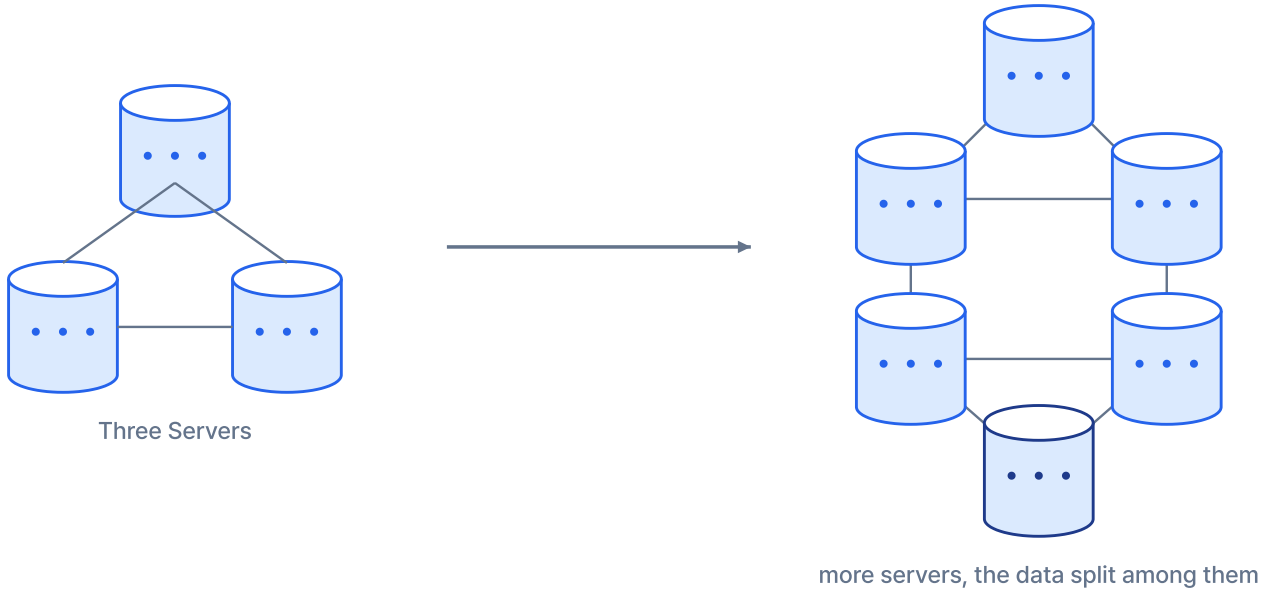
This means improving performance and capacity by adding **more power** (e.g., CPU, RAM, or storage) to a single server.

Scale up: the same server, made bigger

Why NoSQL? – Scalability

Scale out

Distributed SQL and many NoSQL systems scale **horizontally** (also known as **scale-out**).

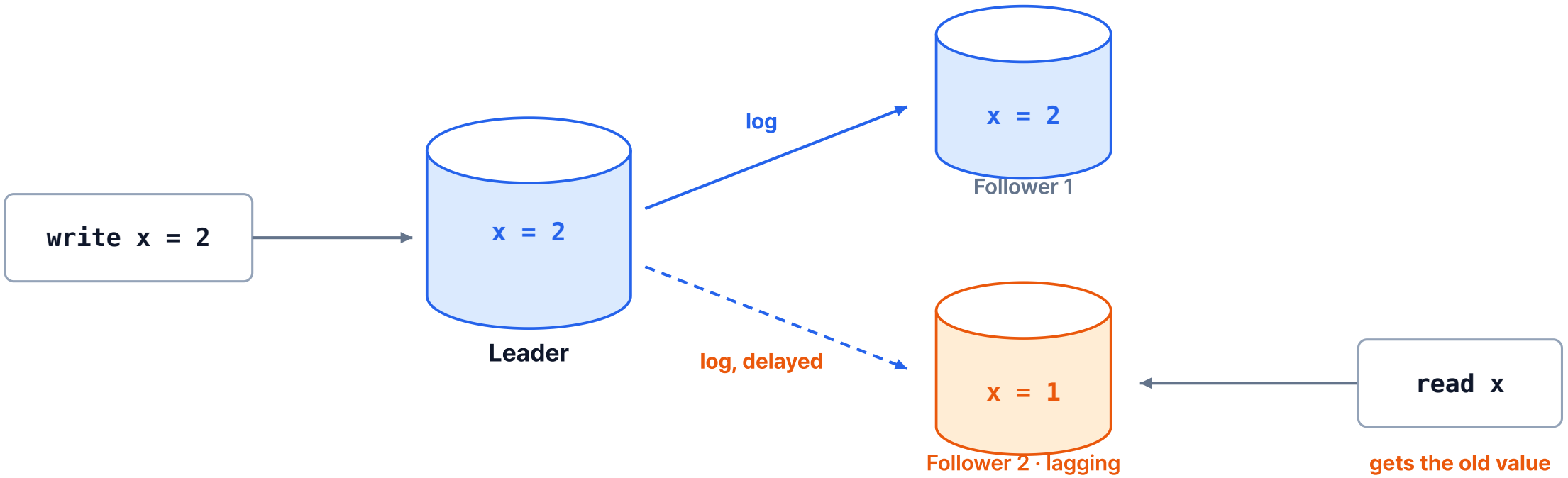


This means increasing capacity by adding **more servers** or nodes to a distributed system.

Scale out: more servers, and the data must be split among them

Asynchronous Leader–Follower Replication

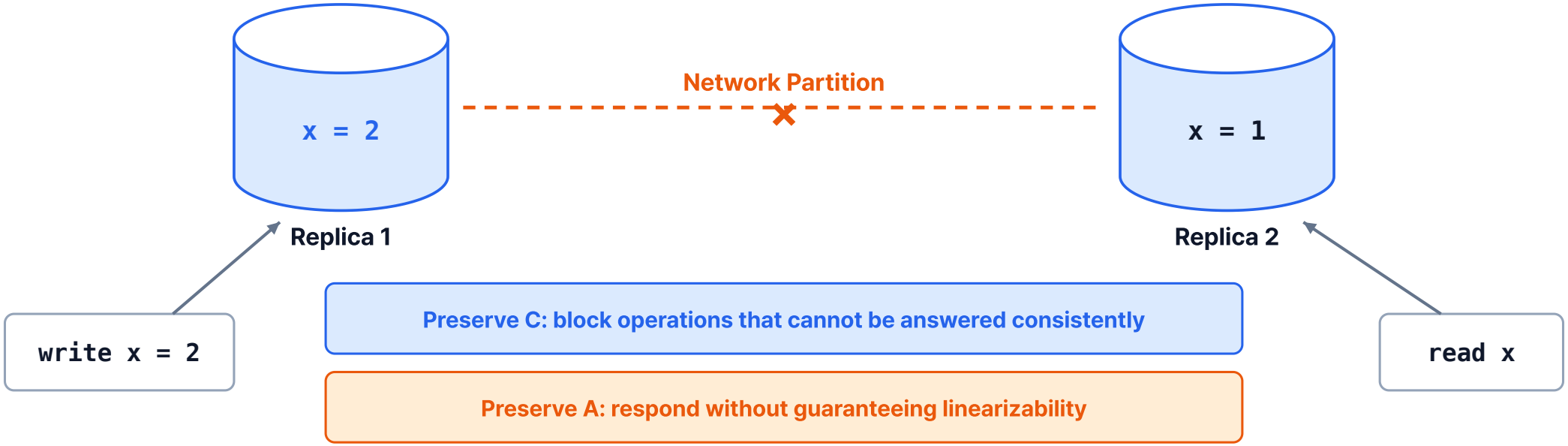
Followers replay the leader's log asynchronously, so a follower read can return an older value



A read from the **lagging follower** returns the old value

CAP Theorem

A partition forces a choice: block some operations, or answer without a consistency guarantee



Without a partition, replication still trades **latency** against **consistency**

During a partition, a system cannot guarantee both **linearizable operations** and **availability**

SQL vs NoSQL

ACID

SQL databases are known for their robust support of **ACID transactions** (Lecture 6).

- **Atomicity**: a transaction is a single, indivisible unit. All of its operations succeed, or none are applied.
- **Consistency**: a transaction takes the database from one valid state to another, keeping constraints, triggers and types intact.
- **Isolation**: the isolation level defines which interactions concurrent transactions may observe.
- **Durability**: once committed, the result survives a crash.

This makes SQL databases ideal where **data integrity** and **consistency** are critical, such as financial systems.

SQL vs NoSQL

BASE

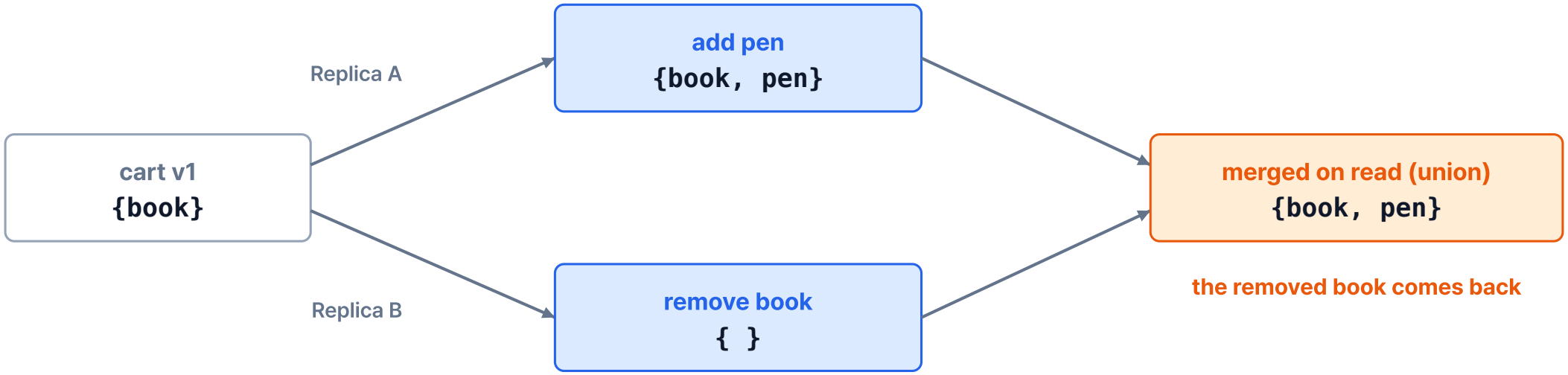
Some distributed systems favor **availability** with eventual consistency, often described as **BASE**:

- **Basically Available**: aim to keep serving requests during failures; success is not unconditional.
- **Soft state**: replicas may differ while updates propagate.
- **Eventually consistent**: replicas converge once writes stop and communication and reconciliation resume.

NoSQL does not imply BASE. MongoDB supports multi-document ACID transactions; guarantees depend on the product and configuration.

Case study: Amazon Dynamo

Dynamo prioritized cart writes during failures and let the application reconcile conflicting versions



Two versions merged by union: a **deleted item can come back**

SQL vs NoSQL

Guarantees depend on the product, the operation and the settings

Product / operation	What is supported
MongoDB multi-document transactions	Atomic changes across several documents
DynamoDB table and LSI reads	Strongly consistent reads can be requested
DynamoDB global secondary index reads	Eventually consistent only
Cassandra writes and reads	Tunable: how many replicas must answer

A database family is **not a consistency setting**: check the product, the operation and the configuration

Model, transaction scope and replica reads are **three separate questions**

Discussion: Which Store Would You Try?

Start with the question, then choose a candidate

Need	Main question
Login sessions	Retrieve a user by session ID
Device readings	Read one device over a time range
Suspicious account links	Follow shared devices and payment links
Product catalog	Query products with different fields

For each row: **one candidate family and one caveat** to check before choosing

Answer: Which Store Would You Try?

Name the question first; the product comes last

Need	Candidate	Check before choosing
Sessions	Key-value	Expiry and persistence
Device readings	Wide-column	Key distribution and group size
Account links	Graph	Path depth and candidate count
Catalog	Document	Indexes and shared updates

These are starting candidates, not exclusive answers: SQL and **SQL with JSON** remain valid for all four

Takeaway: NoSQL I

1. NoSQL is **four data models**, not one technology: key-value, wide-column, graph, document, each with products chosen by role.
2. JSON is a **tree** that describes itself: keys travel with the data, arrays keep their order, and a repeated attribute needs a second table on the way into a relation.
3. The reasons are data model, **schema-less** writes and **scale-out**; scale-out brings lagging replicas, so check each product's consistency and transaction guarantees.